

Introduction to Deep Learning

A reading companion — read this after the lecture,
with the slides beside you

Dr Naila Marir

Effat University · College of Engineering · Computer Science Department

Fall 2026

HOW TO USE THIS DOCUMENT

This follows the lecture in four parts, in the same order as the slides. Where the lecture showed you a figure, the same figure is here — so you can look at it slowly. Every symbol is defined before it is used, and the worked numbers are carried all the way through so you can check each step yourself. Each part ends with a **Check yourself** box: answer those from memory, and you will know whether the part landed.

What this chapter covers

The chapter answers four questions in order. Each one depends on the answer before it, so read them in sequence the first time.

Part 1	Introduction to Deep Learning	what AI, ML and deep learning are, and how a model learns at all
Part 2	What is a Neural Network?	from a single neuron to a network that invents its own features
Part 3	Supervised Learning with Neural Networks	where these models actually earn their keep
Part 4	Why is Deep Learning Taking Off?	the ideas are old — so what changed?
Part 5	Glossary	every term in the chapter, one line each

THE SENTENCE TO CARRY OUT OF THIS CHAPTER

A neural network is not programmed with the rule. It is corrected until its error is small — and the rule is what remains.

NOTATION USED THROUGHOUT · LEARN THIS ONCE

x	one input example — the features we measure
y	the true label — the correct answer
$\hat{y}$ ("y-hat")	the model's prediction . The hat always means <i>estimated</i> , never true
(x, y)	one training example — an input with its correct answer
m	the number of training examples
n_x	the number of features in each example
$x^{(i)}, y^{(i)}$	the i-th example . Round brackets index <i>examples</i>
w, b	the parameters — weights and bias. Training changes only these
$L(\hat{y}, y)$	the loss on a single example
$J(w, b)$	the cost over the whole training set — what we minimise
α ("alpha")	the learning rate — how big a correction step to take

One convention to fix now. A superscript in *round* brackets indexes a training example: $x^{(3)}$ is the third house in the dataset. A superscript in *square* brackets indexes a *layer* of a network: $a^{[3]}$ is the output of layer 3. You will meet layers in Chapter 2; the two are never mixed.

Part 1 Introduction to Deep Learning

AI is the new electricity

About a hundred years ago, electricity transformed nearly every industry in turn — transportation, manufacturing, healthcare, communications. It was not a product; it was a **general-purpose technology**, something so broadly useful that each industry had to rebuild itself around it.

Andrew Ng's claim, and the reason this course exists, is that **AI is now doing the same thing**. That is a claim about breadth, not about any single clever program: the same underlying technique is reshaping medicine, logistics, translation and finance at once.

Three words that are often confused

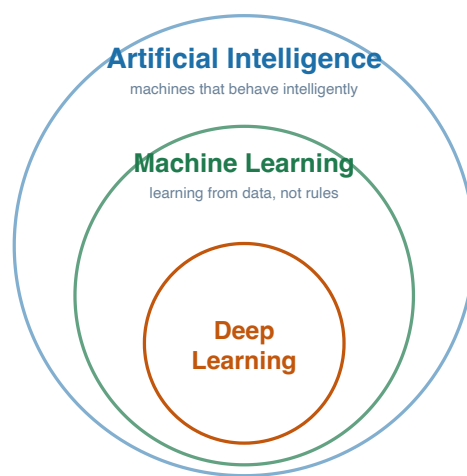


Figure 1. AI contains machine learning, which contains deep learning. The circles are nested, not alternatives.

These three terms are nested, not interchangeable, and being sloppy about them causes real confusion later:

- **Artificial Intelligence** is the widest circle — any technique that makes a machine behave intelligently. A hand-written set of if-then rules counts.
- **Machine Learning** is a subset of AI: the system is not given rules, it *learns* them from data.
- **Deep Learning** is a subset of machine learning: it uses neural networks with many layers, and it learns not only the rule but also *the features the rule uses*.

What "learning" actually means

The definition used across the field is Tom Mitchell's, from 1997. It is worth memorising because it is precise about what must improve:

A computer program is said to **learn** from experience **E** with respect to some task **T** and performance measure **P**, if its performance at **T**, as measured by **P**, **improves with experience E**.

Tom Mitchell, *Machine Learning*, 1997

Abstract on its own, so put the house-price problem into it:

TERM	IN THE HOUSE-PRICE PROBLEM
Experience E	thousands of past houses, each with the price it really sold for
Task T	predict the price of a house the model has never seen
Performance P	how close its predictions are to the real prices

Show the program more houses (more **E**) and its price predictions get closer (better **P**). That improvement *is* the learning. Nothing else about the program changed.

A common misreading. Learning is not "the program stored more houses." A lookup table stores every house and has learned nothing — ask it about a house not in the table and it fails. Learning means performance improves on examples the model has **never seen**. That property has a name: **generalisation**, and it is the real goal of everything in this course.

Three kinds of machine learning

Textbooks usually split machine learning into three families. The clean way to tell them apart is to ask one question: **what signal tells the model it was right?**

- **Supervised learning** — every example arrives with the correct answer attached, as a pair (x , y). The model predicts, compares against y , and corrects itself. Splits into *regression* (the output is a number, like a price) and *classification* (the output is a category, like cat/not-cat).
- **Unsupervised learning** — no labels at all, only x . The model must find structure that is already there: grouping similar customers (*clustering*), or squeezing many features into a few (*dimensionality reduction*).
- **Reinforcement learning** — no correct answer is ever given, only a **reward, later**. A chess engine learns only at the end of the game whether it won, and must work backwards to decide which moves deserved credit.

Almost everything of economic value today — and almost everything in this course — is **supervised** learning.

The training loop

"The algorithm finds the rule" sounds like magic until you see the loop. It is not magic. It is repeated small correction, and it has exactly three steps.

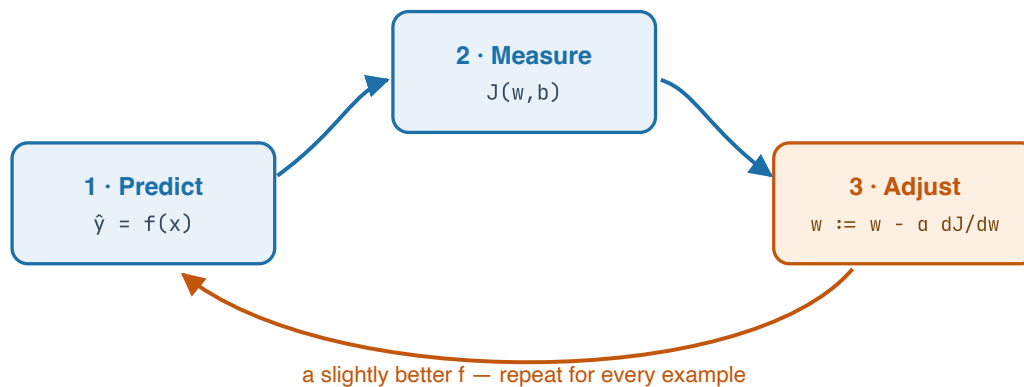


Figure 2. The training loop: predict, measure the error, correct the parameters, then repeat.

1. **Predict.** Take one example x and compute the model's current guess $\hat{y} = f(x)$.
2. **Measure the error.** Compare $\hat{y}$ against the true answer y . This tells us not just *that* we were wrong, but **how wrong** and **in which direction**.
3. **Correct.** Nudge the parameters w and b slightly, in the direction that makes that error smaller.

Then repeat — for every example, and over the whole dataset many times. Notice that this loop *needs* the true y in step 2. That is why the loop belongs to supervised learning: without a correct answer there is no error to measure.

Every ML system has the same four components

Whatever the problem, whatever the algorithm, you can always point at these four parts. Getting used to naming them makes every later topic easier to place.

1 · THE DATASET — WHAT IT LEARNS FROM



Figure 3. The dataset. Each row is one training example: the feature $x^{(i)}$ we measure and the answer $y^{(i)}$ we want.

The dataset is m examples. Each one is a pair: the **features** we measured ($x^{(i)}$) and the **answer** we want to predict ($y^{(i)}$). In the figure the left column is the living area of a house and the right column is its price, so $x^{(3)} = 2400$ and $y^{(3)} = 369$.

2 · THE MODEL AND ITS PARAMETERS — THE SHAPE WE ALLOW

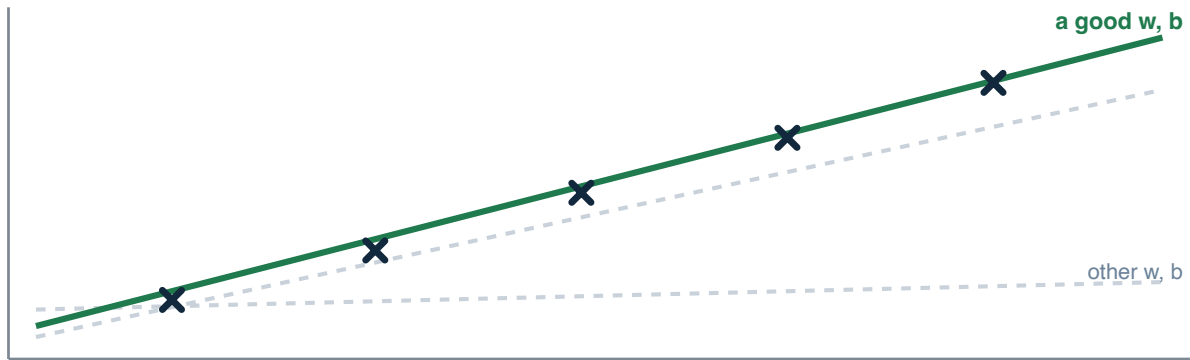


Figure 4. The model is the family of straight lines; the parameters w and b choose which one.

The **model** is the *family* of functions we permit. If we choose straight lines, then the model is "all straight lines" and every candidate has the form:

$$\hat{y} = wx + b$$

The **parameters** w and b pick *which* line out of that infinite family. The engineer chooses the family; training chooses the member. In the figure, the grey dashed lines are other choices of w , b ; the green one fits the data.

3 · THE COST FUNCTION — HOW WRONG WE ARE, IN ONE NUMBER

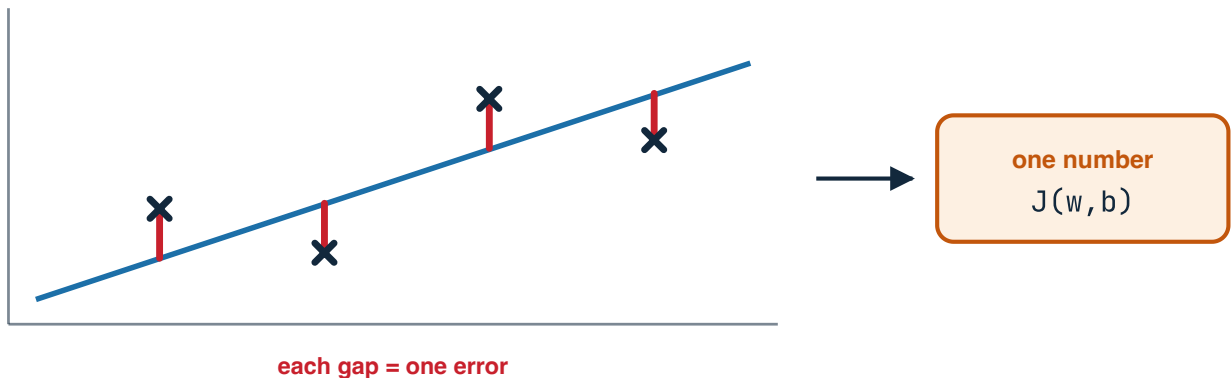


Figure 5. Every point has a gap to the line. The cost squares those gaps and averages them into one number.

For a single example the gap between prediction and truth is $\hat{y}^{(i)} - y^{(i)}$. We need one number for the *whole* dataset, so we square each gap and average them:

$$J(w, b) = (1/m) \sum_{i=1..m} (\hat{y}^{(i)} - y^{(i)})^2$$

the mean squared error

Why square? Two reasons, both practical. First, a prediction that is 50 too low and one that is 50 too high are equally bad — squaring makes both count as positive, so they cannot cancel out. Second, squaring punishes large misses much more than small ones, which is usually what we want.

Keep L and J apart. $L(\hat{y}, y)$ is the error on *one* example. $J(w, b)$ is the average over *all* m of them. Training minimises J .

4 · THE LEARNING ALGORITHM — HOW IT IMPROVES



Figure 6. Gradient descent: start anywhere on the cost curve and step downhill until it stops falling.

We now have a number, $J(w, b)$, that says how bad the current parameters are. The learning algorithm changes w and b to make it smaller. The standard method is **gradient descent**:

$$w := w - \alpha \cdot \partial J / \partial w \quad b := b - \alpha \cdot \partial J / \partial b$$

Read it as "step downhill". The derivative $\partial J / \partial w$ is the **slope** of the cost curve — it says which direction is downhill and how steep it is. The minus sign means we move *against* the slope. The **learning rate** α controls how big each step is: too small and training crawls, too large and it can overshoot the bottom entirely.

Worked example — one full correction step

PREDICTING HOUSE PRICE FROM FLOOR AREA

Suppose the model currently holds $w = 2000$ and $b = 50000$, and we meet one training example: a 100 m² house that really sold for 300,000.

QUANTITY	SYMBOL	VALUE
Input — floor area	x	100
True price	y	300,000
Prediction	$\hat{y} = wx + b$	$2000(100) + 50000 = 250,000$
Error	$\hat{y} - y$	$250,000 - 300,000 = -50,000$

The error is **negative**, which tells us the prediction was *too low*. So w should increase. Nudge it to $w = 2050$ and this house now predicts 255,000 — still low, but closer. Repeat across thousands of houses and w and b settle at values that price houses well *in general*.

Notice what never happened: **nobody wrote the pricing rule**. No one told the system "price rises by roughly 3,000 per square metre". That relationship was *discovered*, purely by making an error smaller over and over.

CHECK YOURSELF

1. State Mitchell's definition, and say what E , T and P are for spam filtering. — *E : emails already marked spam / not spam. T : label a new email. P : how many it labels correctly.*
2. What is the difference between L and J ? — *L is the error on one example; J is the average over all m examples.*
3. Why do we square the errors in the cost function? — *So too-high and too-low misses don't cancel, and so large errors are punished more.*
4. In the worked example the error was $-50,000$. Should w go up or down, and why? — *Up: a negative error means the prediction was below the true price.*
5. Which of the four components does the engineer choose, and which does training choose? — *The engineer chooses the dataset, the model family and the cost function; training chooses the parameters w and b .*

Part 2 What is a Neural Network?

The phrase "neural network" sounds like it requires a whole new theory. It does not. Everything in Part 1 was already, in effect, a neural network with exactly one neuron. This part builds up from that.

A single neuron

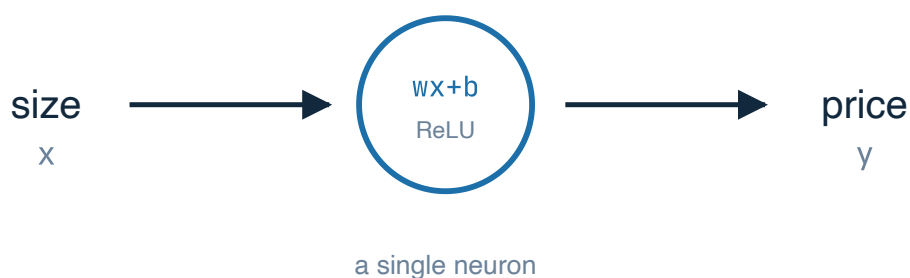


Figure 7. A single neuron. Size goes in, $w x + b$ is computed, a price comes out.

Go back to the house-price example. The size of the house goes in, some arithmetic happens, a price comes out. Draw that arithmetic as a circle and you have drawn a **neuron**:

$$x \rightarrow (wx + b) \rightarrow \hat{y}$$

That is the whole idea. A neuron takes inputs, multiplies each by a weight, adds a bias, and produces an output. The little function applied at the end is called the **activation function**.

RELU — THE ACTIVATION YOU WILL SEE MOST

A house price can never be negative, so a sensible neuron should output zero rather than a negative price. The function that does this is **ReLU**, the *Rectified Linear Unit*:

$$\text{ReLU}(z) = \max(0, z)$$

flat at zero for negative input, then a straight line

It is deliberately simple — "take the bigger of zero and the input" — and that simplicity turns out to matter enormously for training speed. We return to why in Part 4.

Stacking neurons into a network

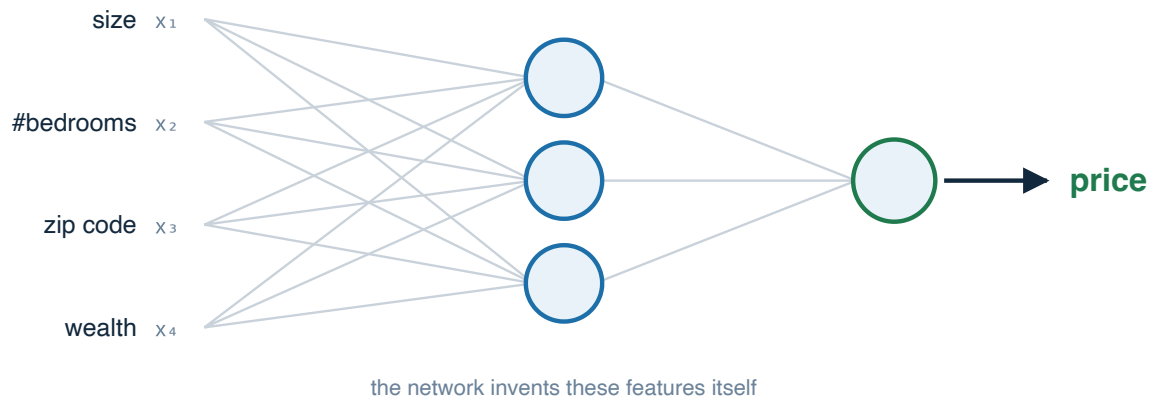


Figure 8. A network with a hidden layer. Nobody tells the middle neurons what to represent — they work it out.

One neuron with one feature is limited. A real house has a size, a number of bedrooms, a postcode, and a neighbourhood wealth level. So we use several input features, and — this is the key move — we put a **layer of neurons in the middle**.

Every input connects to every middle neuron, and those connect to the output. The middle column is called a **hidden layer**: hidden because you never supply its values and never read them out, you only see x going in and $\hat{y}$ coming out.

THE PROPERTY THAT MAKES DEEP LEARNING DIFFERENT

You never tell the hidden neurons what to represent. Given enough examples of (features $\rightarrow$ price), they work out useful intermediate quantities *on their own*.

In the housing case, the network may end up with one hidden neuron that effectively measures *family size* (from size and bedrooms), another that measures *walkability* (from the postcode), and another that measures *school quality* (from postcode and wealth). Nobody designed those. Nobody labelled them. They emerged because they were useful for reducing the price error.

Why this is such a big deal. Before deep learning, a human expert had to invent those intermediate quantities by hand — a slow, brittle process called **feature engineering**, which had to start over for every new task. Deep learning removes that step: the features are *learned, not designed*.

The word "deep"

"Deep" simply means **many hidden layers** rather than one. Depth is what produces a **hierarchy of features**: early layers pick up simple things, later layers combine them into complicated things. Trained on faces, a network typically learns edges in its first layer, then eyes and noses in the middle, then whole faces near the output — each layer built from the one before it.

Part 3 Supervised Learning with Neural Networks

Almost all of the economic value created by neural networks so far has come from one setting: **supervised learning**, where you have inputs x paired with correct answers y , and you want a function that maps one to the other.

INPUT (X)	OUTPUT (Y)	APPLICATION	NETWORK
Home features	Price	Real estate	Standard NN
Ad, user info	Click? (0/1)	Online advertising	Standard NN
Image	Object (1...1000)	Photo tagging	CNN
Audio	Text transcript	Speech recognition	RNN
English	Arabic	Machine translation	RNN
Image, radar	Position of cars	Autonomous driving	Hybrid

Figure 9. Supervised learning in practice. Every row is the same pattern: assemble (x, y) pairs, then train a network to map one to the other.

Read that table as a single pattern repeated. In every row somebody assembled a dataset of (x, y) pairs and trained a network to map x to y . Only the data changes.

Different data, different architecture

The right-hand column of the table names the kind of network used, and the choice is not arbitrary — it follows from the *shape* of the input:

- **Standard neural networks** for tabular data such as house features or advertising records, where each input is a plain list of numbers.
- **Convolutional Neural Networks (CNNs)** for *images*. A CNN is built around the fact that pixels near each other are related, and that a cat is a cat wherever it appears in the frame. Weeks 7–9
- **Recurrent Neural Networks (RNNs)** for *sequences* — audio, text, anything with a time order, where what came earlier changes the meaning of what comes next. Week 10
- **Hybrid or custom architectures** for complex inputs such as autonomous driving, which fuses camera images with radar.

Structured and unstructured data

This distinction explains where deep learning changed things most.

- **Structured data** is a table. Each column has a name and a meaning: size in m^2 , number of bedrooms, user age. Each feature is already meaningful on its own.
- **Unstructured data** is raw pixels, audio samples, or text. A single pixel value means nothing by itself; the meaning lives in the pattern across thousands of them.

Humans are extraordinarily good at unstructured data — you recognise a face or a spoken word effortlessly. Computers historically were terrible at it. Deep learning is what closed that gap, and it is why the headline results are in vision, speech and language.

Do not over-read this. On structured, tabular data, classical methods such as gradient boosting remain extremely strong and are often the first thing a practitioner should try. Deep learning is not automatically the right tool; it is the right tool when the input is raw and high-dimensional.

CHECK YOURSELF

1. What makes an application "supervised"? — *The training data contains the correct answer y for every input x .*
2. Why is a CNN suited to images rather than a standard network? — *It is built for the fact that nearby pixels are related and that an object keeps its identity wherever it appears in the image.*
3. Give an example of structured and of unstructured data. — *Structured: a table of house size and price.
Unstructured: the photograph of the house.*

Part 4 Why is Deep Learning Taking Off?

Here is the puzzle. Neural networks are not new — the core mathematics dates to the 1950s–80s, and backpropagation was popularised in 1986. So why did almost nothing happen for decades, and then everything happened at once after about 2010?

The answer is **scale**, and the clearest way to see it is this figure.

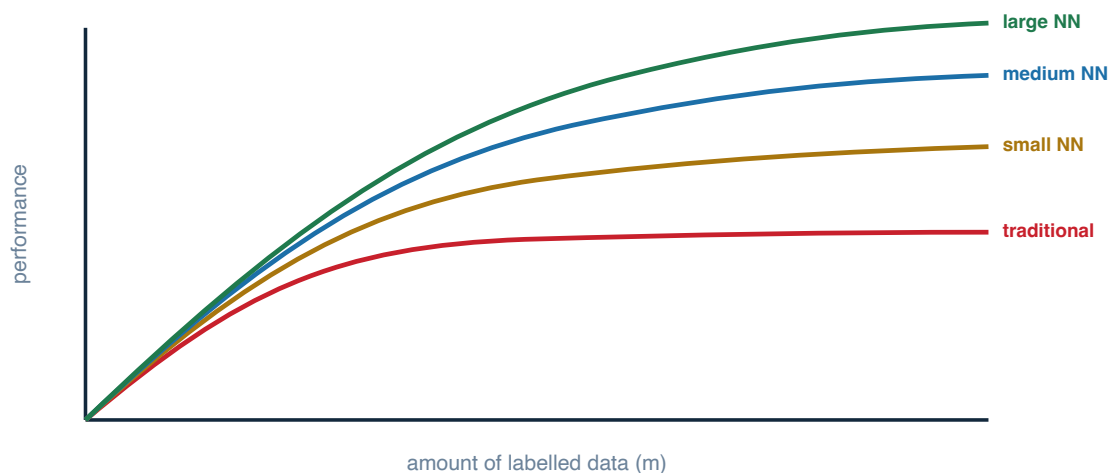


Figure 10. Scale drives progress. Traditional algorithms plateau; larger networks keep improving as data grows.

Reading the scale curve

The horizontal axis is the amount of **labelled data** available (m). The vertical axis is **performance**. There are four curves, and the story is in their shapes:

- **Traditional algorithms** (red) — support vector machines, logistic regression. They improve as data grows, then **plateau**. Past a certain point, feeding them more data buys you almost nothing.
- **Small, medium and large neural networks** (yellow, blue, green) — each keeps climbing for longer, and the larger the network, the higher it eventually reaches.

Two consequences follow, and both matter for how you work:

WHAT THE CURVE ACTUALLY TELLS YOU

1. **To get the best performance you need two things at once** — a big enough network *and* enough data. A large network trained on a small dataset does not help; the green curve only pulls ahead on the right-hand side of the graph.
2. **On the far left, the ordering is unreliable.** With very little data, a well-engineered traditional algorithm can easily beat a neural network. Which method wins is down to skill and hand-crafted features, not to the method itself. Deep learning's dominance is a large-data phenomenon.

The three drivers

So what moved us to the right-hand side of that graph? Three things arrived together, and all three were necessary.

- **Data.** Society digitised. Activity moved onto computers and phones, so enormous labelled datasets became possible for the first time.
- **Computation.** GPUs, and specialised hardware after them, made it feasible to train a large network in hours instead of months.
- **Algorithms.** Better ideas made training faster — and, importantly, most of these were *aimed at* speed.

ONE ALGORITHMIC CHANGE WORTH UNDERSTANDING: SIGMOID → RELU

Early networks used the **sigmoid** activation, an S-shaped curve. Its problem is visible in its shape: at both ends the curve goes almost flat, so its slope is nearly zero. And gradient descent moves using the slope. When the slope vanishes, learning effectively **stalls**.

ReLU — $\max(0, z)$ — has a constant slope of 1 for all positive inputs. The gradient does not vanish, so gradient descent keeps making real progress. Swapping one function for another made training dramatically faster, with no new hardware required.

Why faster training matters more than it sounds

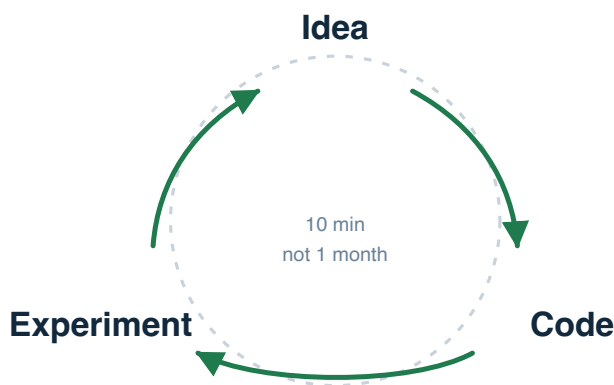


Figure 11. Faster training means more turns of the idea → code → experiment loop, and more turns means better models.

Speed is not just convenience. Research and engineering both run as a loop: you have an **idea**, you write the **code**, you run the **experiment**, and the result gives you your next idea.

If one turn of that loop takes ten minutes, you can try dozens of ideas in a day. If it takes a month, you might try ten in a year — and you will choose conservatively, because each attempt is expensive. **Faster computation buys more iterations, and more iterations buy better models.** That is the real reason hardware progress translated so directly into better AI.

CHECK YOURSELF

1. Why did neural networks not "take off" in the 1990s, given the maths already existed? — *Not enough labelled data and not enough computing power to train large networks.*
2. On the scale curve, why does the red traditional-algorithm line flatten? — *Those methods cannot exploit extra data beyond a point; their capacity is limited.*
3. Why does ReLU train faster than sigmoid? — *Sigmoid's slope goes almost to zero at both ends so learning stalls; ReLU has slope 1 for all positive inputs.*

4. If you have only 200 labelled examples, is a very large network the right choice? — *Probably not — on the left of the curve a simpler method with good features is often better.*

Activation function

The small function applied at a neuron's output, such as ReLU or sigmoid.

Artificial Intelligence (AI)

Any technique that makes a machine behave intelligently, including hand-written rules.

Backpropagation

The algorithm that computes how much each parameter contributed to the error. Chapter 2.

Bias, b

The parameter added after the weighted inputs; it shifts the output up or down.

Classification

Supervised learning where the output is a category.

Clustering

Unsupervised learning that groups similar examples with no labels.

CNN

Convolutional Neural Network — the architecture used for images. Weeks 7–9.

Cost function, $J(w,b)$

The average loss over the whole training set; the single number training minimises.

Deep learning

Machine learning with neural networks of many layers, which learn their own features.

Dimensionality reduction

Unsupervised learning that compresses many features into fewer.

Feature

One input variable describing an example — one of the numbers in x .

Feature engineering

Designing input variables by hand. The step deep learning removes.

Generalisation

Performing well on new, unseen data. The actual goal of training.

Gradient descent

Repeatedly stepping the parameters in the direction that reduces the cost.

Hidden layer

A layer between input and output whose values you neither supply nor read.

Label

The correct answer y attached to a training input x .

Learning rate, α

How large a step gradient descent takes. Too small is slow; too large overshoots.

Loss, $L(\hat{y}, y)$

How wrong a single prediction was.

Machine Learning

A subset of AI in which the rule is learned from data instead of written by hand.

Mean squared error

A cost function that squares each error and averages them.

Model

The family of functions allowed, e.g. all straight lines $\hat{y} = wx + b$.

Neuron

A unit that weights its inputs, adds a bias, and applies an activation function.

Overfitting

Fitting the training data so closely that performance on new data suffers. Revisited later.

Parameters

The numbers inside the model that training adjusts: the weights w and bias b .

Regression

Supervised learning where the output is a continuous number.

Reinforcement learning

Learning from delayed, sparse rewards rather than labelled answers.

ReLU

Rectified Linear Unit, $\max(0, z)$. The most common activation function.

RNN

Recurrent Neural Network — the architecture used for sequences. Week 10.

Sigmoid

An older S-shaped activation whose flat ends make training stall.

Structured data

Data in a table, where each column has a defined meaning.

Supervised learning

Learning from labelled pairs (x, y) .

Training example

One pair $(x^{(0)}, y^{(0)})$ from the dataset.

Unstructured data

Raw pixels, audio or text, where no single value is meaningful alone.

Unsupervised learning

Learning structure from data that has no labels.

Weight, w

The parameter multiplying an input; it sets how much that input matters.